Temperature Data Analysis API Assignment

Problem Statement

You are given data from the Berkeley Earth dataset containing historical temperature records for major cities worldwide. The dataset (`GlobalLandTemperaturesByMajorCity.csv`) contains monthly temperature measurements from 1849 onwards with columns for date, average temperature, uncertainty, city, country, and coordinates.

Using the provided helper functions for basic data extraction, implement five analytical functions:

1. **`find_temperature_extremes()`** - Identify hottest and coldest months on record for a city
2. **`get_seasonal_averages()`** - Calculate seasonal temperature patterns (spring, summer, fall, winter)

3. **`compare_decades()`** - Perform climate change analysis by comparing temperature averages between decades
4. **`find_similar_cities()`** - Find cities with similar climate profiles based on temperature tolerance
5. **`get_temperature_trends()`** - Analyze long-term temperature trends using moving averages and identify warming/cooling periods

Note that some of these are not very well defined: there are different ways to compute moving averages for example. You can choose a suitable approach. In general, simple approaches are preferred - the goal of this assignment is only to see how you approach the problem - complicated solutions with excessive error checking are neither necessary nor desirable.

Dataset Format

The CSV file contains the following columns: - `dt`: Date in YYYY-MM-DD format - `AverageTemperature`: Monthly average temperature in Celsius - `AverageTemperatureUncertainty`: Temperature measurement uncertainty - `City`: City name - `Country`: Country name
- `Latitude`: Geographic latitude - `Longitude`: Geographic longitude

Implementation Requirements

Students must: - Use only Python standard library and csv module. In particular, a lot of this would be very easy using `pandas` - this is not allowed for this assignment. Write any helper functions you need for computing mean, variance etc. - Handle missing or invalid temperature data gracefully - Follow

the exact function signatures and return formats specified in `city_temperature_function.py` - Implement proper error handling for edge cases (missing cities, insufficient data). However, do not go overboard with error handling - only basic checks are expected. Making the code very long with too many checks will lose marks for clarity. - Write clean, readable code with appropriate documentation

Evaluation Framework

You need to submit the following:

1. A single Python file containing your code. You can add other helper functions here if needed. It should be possible to import the functions from this file into another file for testing, so only functions should be here, and the `main()` function should be under a `if` check as shown.
2. A single PDF file with a brief report of what you have done for each function. The file can contain simple plots of the temperature of a given city as demonstration of the `get_city_temperatures` function. Explanations should be brief, and the entire report should be not more than 2 pages long including figures.

The code will be tested against separate test cases, and you *may* be asked to meet the TAs to explain part or all of your solution.

Scoring Breakdown

- **Functionality (60%)**: Correct implementation of all five functions with accurate mathematical calculations
- **Code Quality & Documentation (25%)**: Clean code structure, appropriate comments, and adherence to Python best practices
- **Error Handling Robustness (15%)**: Proper handling of edge cases, missing data, and invalid inputs

Test Categories

- **Basic Functionality**: Verify correct data parsing and basic calculations
- **Mathematical Accuracy**: Validate statistical calculations and trend analysis algorithms

- **Edge Case Handling**: Test behavior with missing cities, insufficient data, and boundary conditions
- **Data Quality Scenarios**: Handle malformed data and missing temperature values
- **Performance Benchmarks**: Ensure reasonable performance on large datasets
- **Return Format Validation**: Verify exact adherence to specified dictionary structures

Example Test Scenarios

- Temperature extremes for cities with complete vs. incomplete data
- Seasonal averages with varying data availability across years
- Decade comparisons with overlapping and non-overlapping periods
- Similar city matching with different tolerance levels
- Trend analysis for cities with different data ranges and patterns